

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [3]: data = pd.read_csv(r"house_price_dataset.csv")
data.head()
```

```
Out[3]:
```

	Square_Footage	Num_Bedrooms	Num_Bathrooms	Year_Built	Lot_Size	Garage_Size	N
0	1360	2	1	1981	0.599637	0	
1	4272	3	3	2016	4.753014	1	
2	3592	1	2	2016	3.634823	0	
3	966	1	2	1977	2.730667	1	
4	4926	2	1	1993	4.699073	0	

```
In [4]: data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Square_Footage         1000 non-null   int64
1   Num_Bedrooms           1000 non-null   int64
2   Num_Bathrooms          1000 non-null   int64
3   Year_Built              1000 non-null   int64
4   Lot_Size                1000 non-null   float64
5   Garage_Size             1000 non-null   int64
6   Neighborhood_Quality    1000 non-null   int64
7   House_Price             1000 non-null   float64
dtypes: float64(2), int64(6)
memory usage: 62.6 KB
```

```
In [5]: data.isna().sum()
```

```
Out[5]: Square_Footage      0
Num_Bedrooms      0
Num_Bathrooms      0
Year_Built         0
Lot_Size           0
Garage_Size        0
Neighborhood_Quality  0
House_Price        0
dtype: int64
```

```
In [6]: data.describe()
```

Out[6]:

	Square_Footage	Num_Bedrooms	Num_Bathrooms	Year_Built	Lot_Size	Garag
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.0
mean	2815.422000	2.990000	1.973000	1986.550000	2.778087	1.0
std	1255.514921	1.427564	0.820332	20.632916	1.297903	0.8
min	503.000000	1.000000	1.000000	1950.000000	0.506058	0.0
25%	1749.500000	2.000000	1.000000	1969.000000	1.665946	0.0
50%	2862.500000	3.000000	2.000000	1986.000000	2.809740	1.0
75%	3849.500000	4.000000	3.000000	2004.250000	3.923317	2.0
max	4999.000000	5.000000	3.000000	2022.000000	4.989303	2.0

In [7]:

```
data.corr()
```

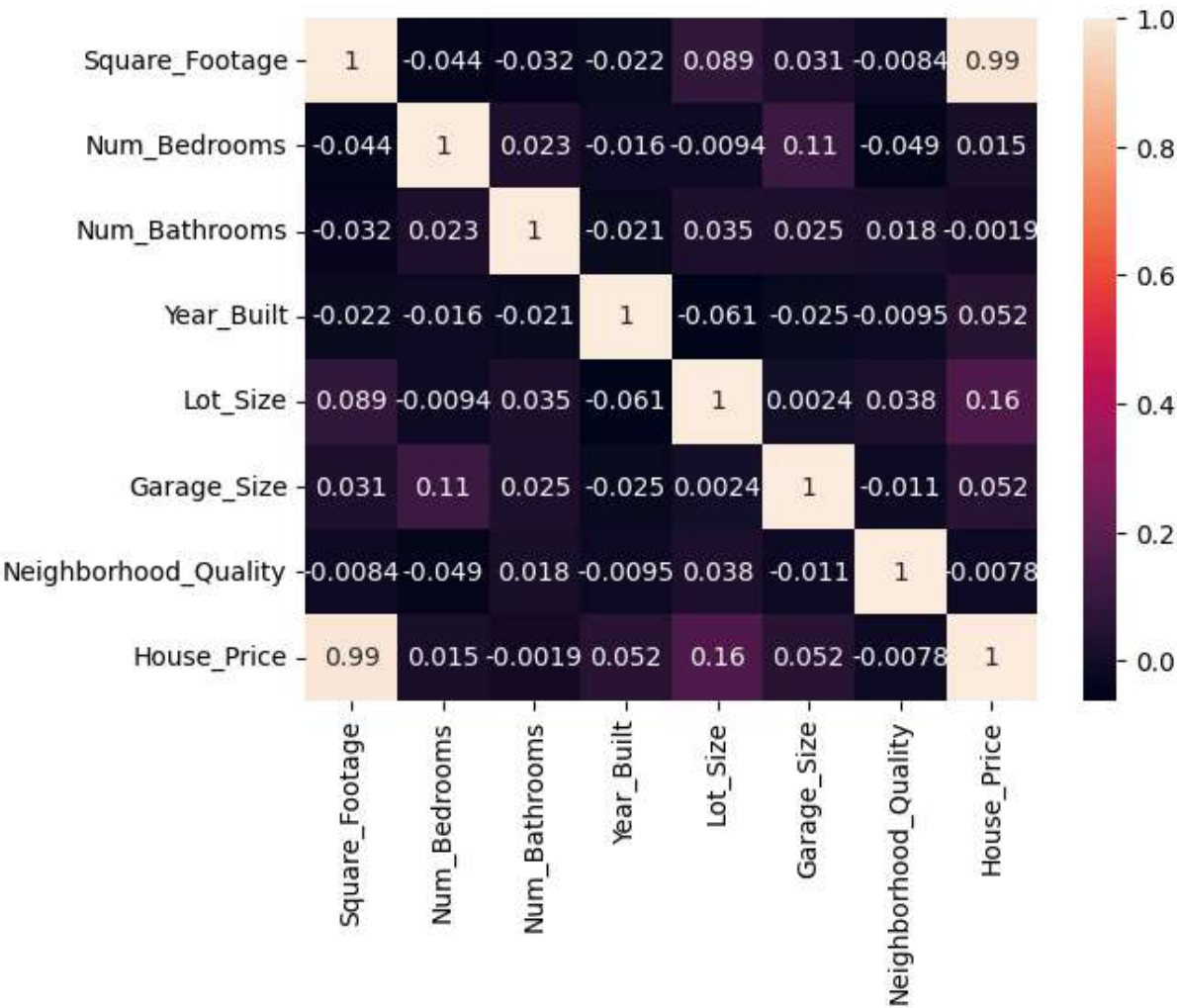
Out[7]:

	Square_Footage	Num_Bedrooms	Num_Bathrooms	Year_Built	Lot_Size
Square_Footage	1.000000	-0.043564	-0.031584	-0.022392	0.08
Num_Bedrooms	-0.043564	1.000000	0.022848	-0.015820	-0.00
Num_Bathrooms	-0.031584	0.022848	1.000000	-0.021063	0.03
Year_Built	-0.022392	-0.015820	-0.021063	1.000000	-0.06
Lot_Size	0.089479	-0.009355	0.034923	-0.061050	1.00
Garage_Size	0.030593	0.113761	0.024846	-0.025485	0.00
Neighborhood_Quality	-0.008357	-0.049024	0.017585	-0.009549	0.03
House_Price	0.991261	0.014633	-0.001862	0.051967	0.16

In [8]:

```
sns.heatmap(data.corr(), annot=True)
```

Out[8]: <Axes: >



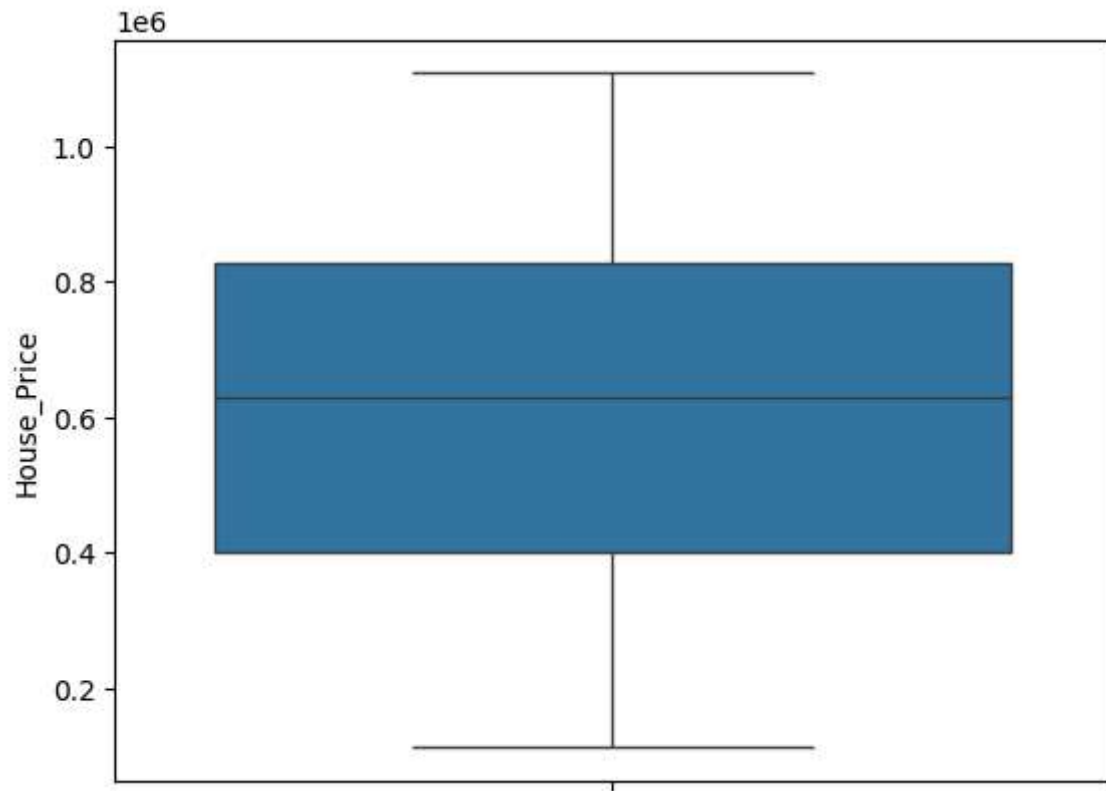
```
In [9]: data.head(2)
```

Out[9]:

	Square_Footage	Num_Bedrooms	Num_Bathrooms	Year_Built	Lot_Size	Garage_Size	N
0	1360	2		1981	0.599637	0	
1	4272	3		2016	4.753014	1	

```
In [10]: sns.boxplot(y='House_Price', data=data)
```

Out[10]: <Axes: ylabel='House_Price'>



```
In [11]: X = data.drop('House_Price', axis=1)
X.head(2)
```

```
Out[11]:
```

	Square_Footage	Num_Bedrooms	Num_Bathrooms	Year_Built	Lot_Size	Garage_Size	N
0	1360	2	1	1981	0.599637	0	
1	4272	3	3	2016	4.753014	1	

```
In [14]: pd.set_option('display.float_format', '{:,.4f}'.format)
```

```
In [15]: y = data['House_Price']
y
```

```
Out[15]:
```

0	262382.8523
1	985260.8545
2	777977.3901
3	229698.9187
4	1041740.8589
...	
995	701493.9971
996	683723.1607
997	572024.0236
998	964865.2986
999	742599.2533

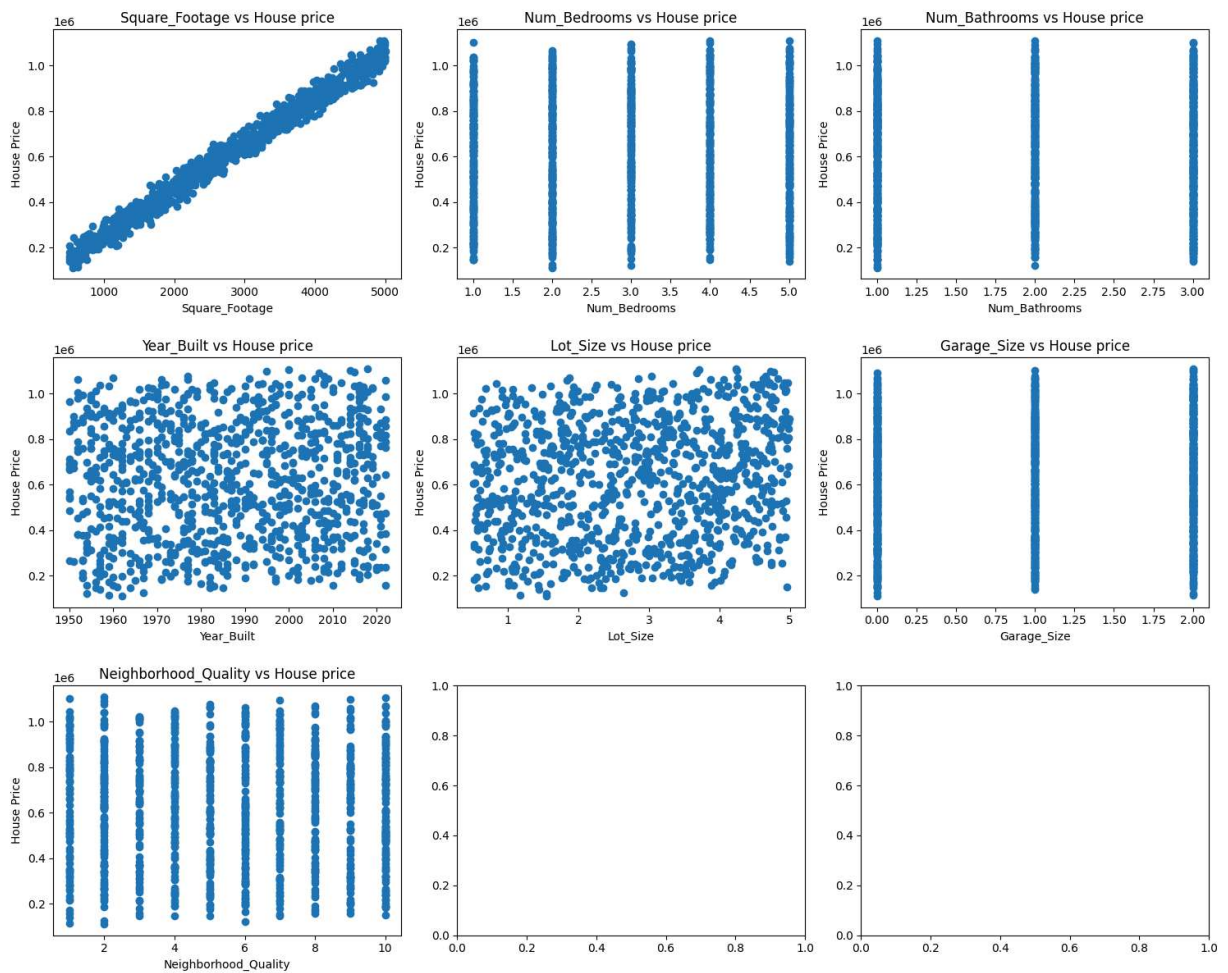
Name: House_Price, Length: 1000, dtype: float64

```
In [26]: import math
fig_count = len(X.columns)
```

```

col = 3
row = math.ceil(fig_count / col)
fig, axes = plt.subplots(row, col, figsize=(15,row * 4))
axes = axes.flatten()
for i, col in enumerate(X.columns):
    axes[i].scatter(X[col], y)
    axes[i].set_title(f"{col} vs House price")
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('House Price')
plt.tight_layout()
plt.show()

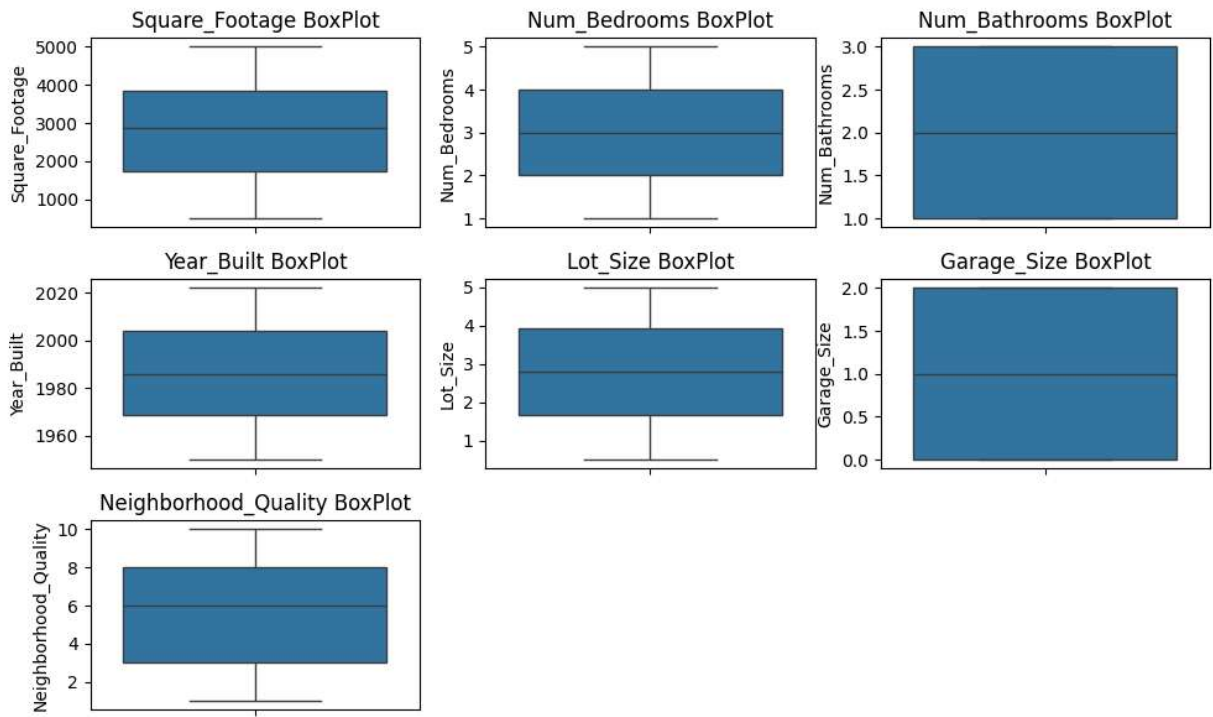
```



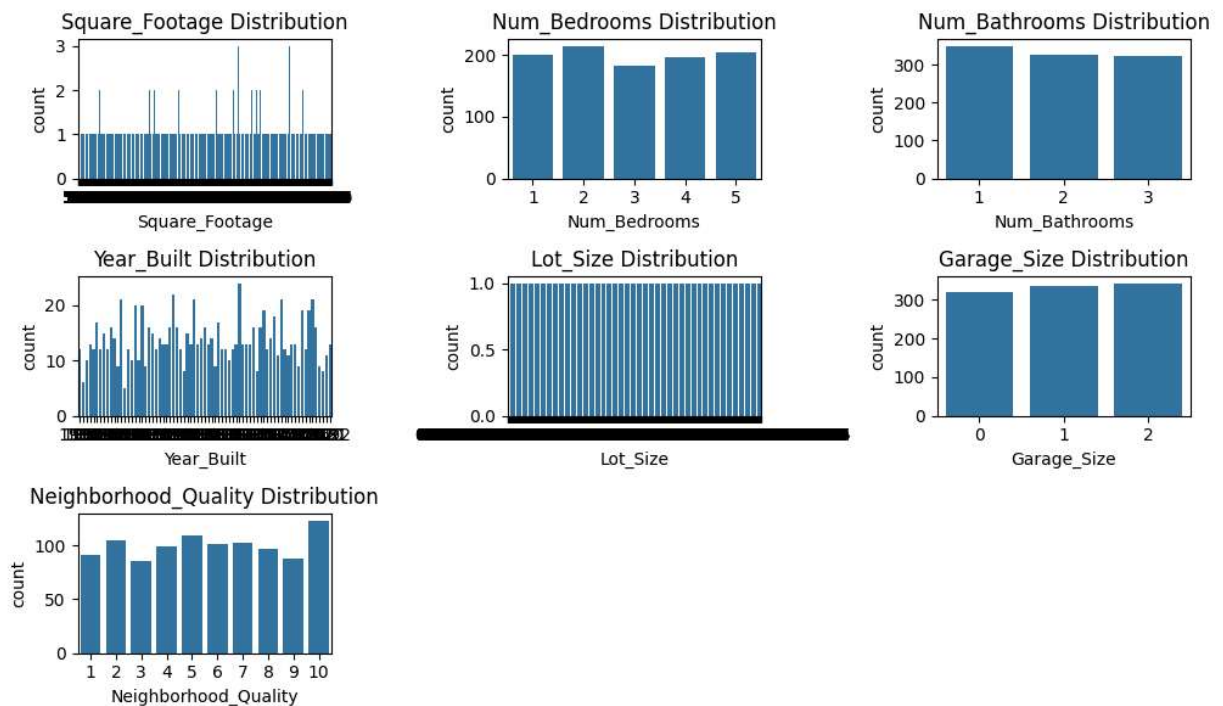
```

In [34]: plt.figure(figsize=(10,6))
for i, column in enumerate(X.columns):
    plt.subplot(3,3,i+1)
    sns.boxplot(X[column])
    plt.title(f"{column} BoxPlot")
plt.tight_layout()
plt.show()

```



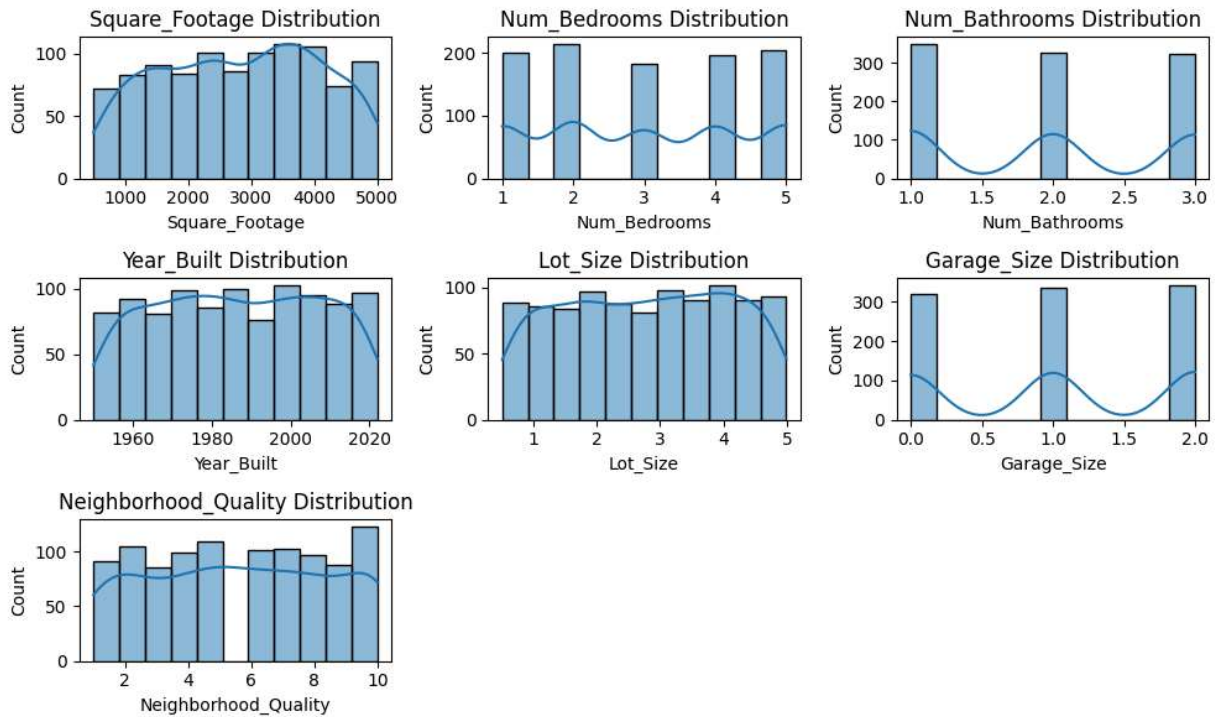
```
In [39]: plt.figure(figsize=(10,6))
for i, column in enumerate(X.columns):
    plt.subplot(3,3, i+1)
    sns.barplot(data[column].value_counts())
    plt.title(f"{column} Distribution")
plt.tight_layout()
plt.show()
```



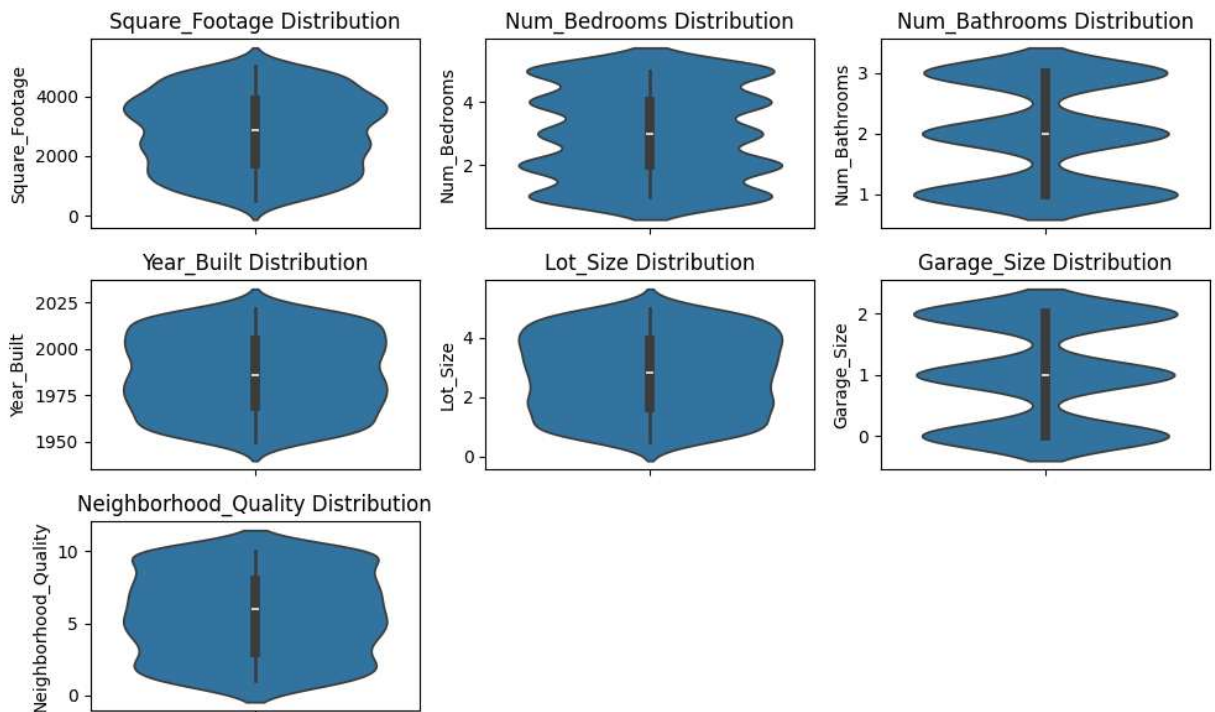
```
In [43]: plt.figure(figsize=(10,6))
for i, column in enumerate(X.columns):
    plt.subplot(3,3,i+1)
    sns.histplot(data[column], kde=True)
```



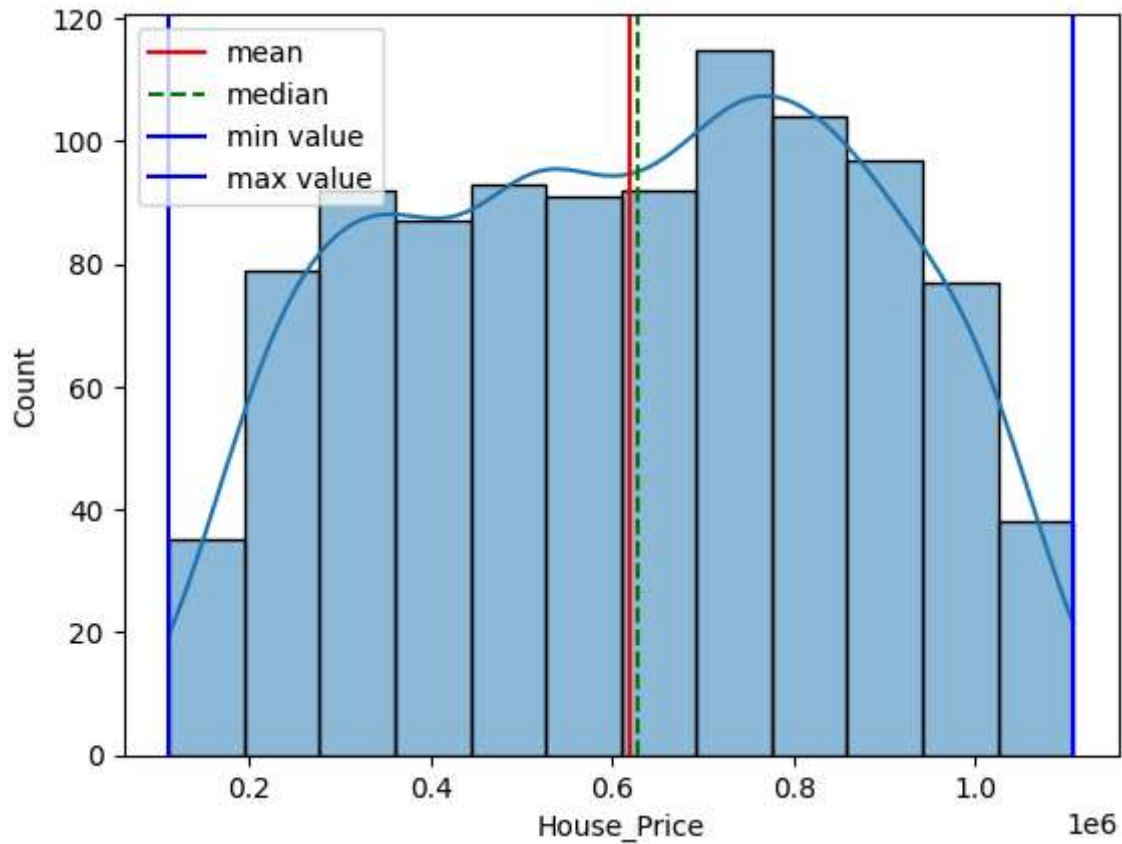
```
plt.title(f"{column} Distribution")
plt.tight_layout()
plt.show()
```



```
In [45]: plt.figure(figsize=(10,6))
for i, column in enumerate(X.columns):
    plt.subplot(3,3,i+1)
    sns.violinplot(data[column])
    plt.title(f"{column} Distribution")
plt.tight_layout()
plt.show()
```

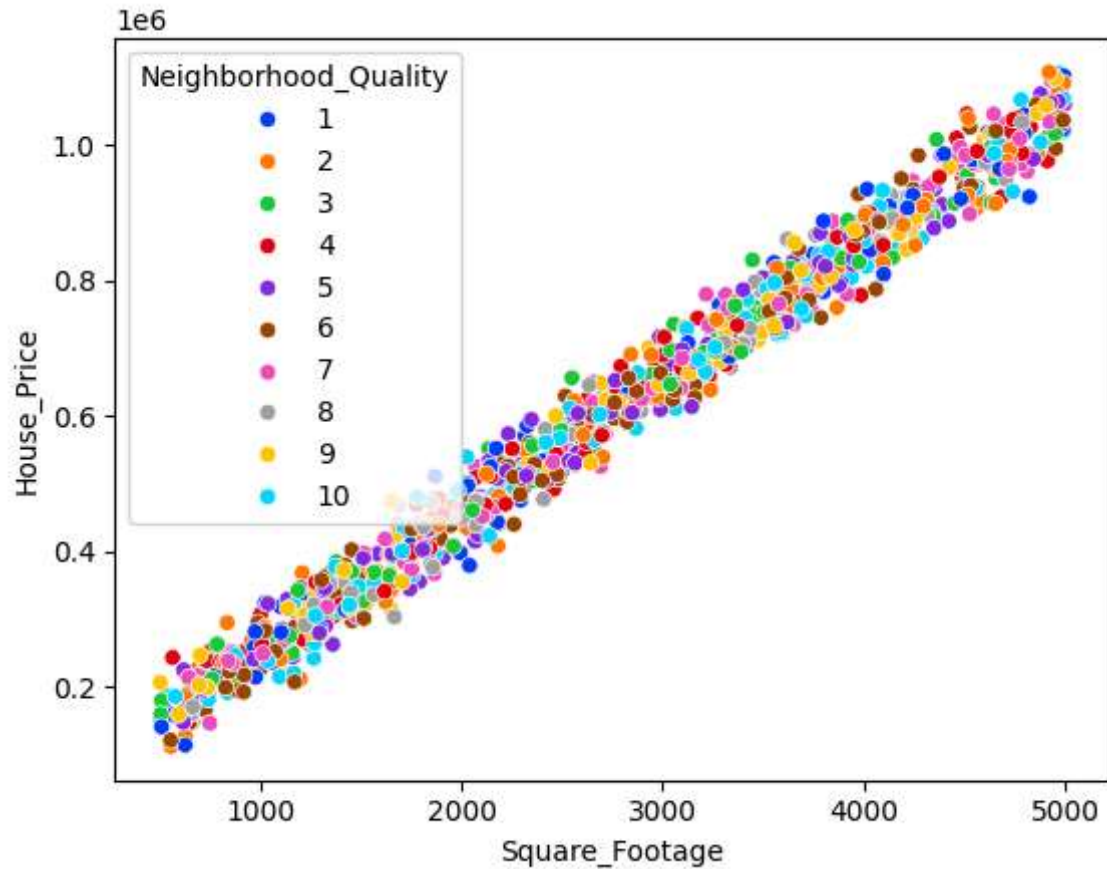


```
In [52]: sns.histplot(x='House_Price', data=data, kde=True)
plt.axvline(data.House_Price.mean(), color='red', label='mean')
plt.axvline(data.House_Price.median(), color='green', linestyle='--', label='median')
plt.axvline(data.House_Price.min(), color='blue', label='min value')
plt.axvline(data.House_Price.max(), color='blue', label='max value')
plt.legend()
plt.show()
```



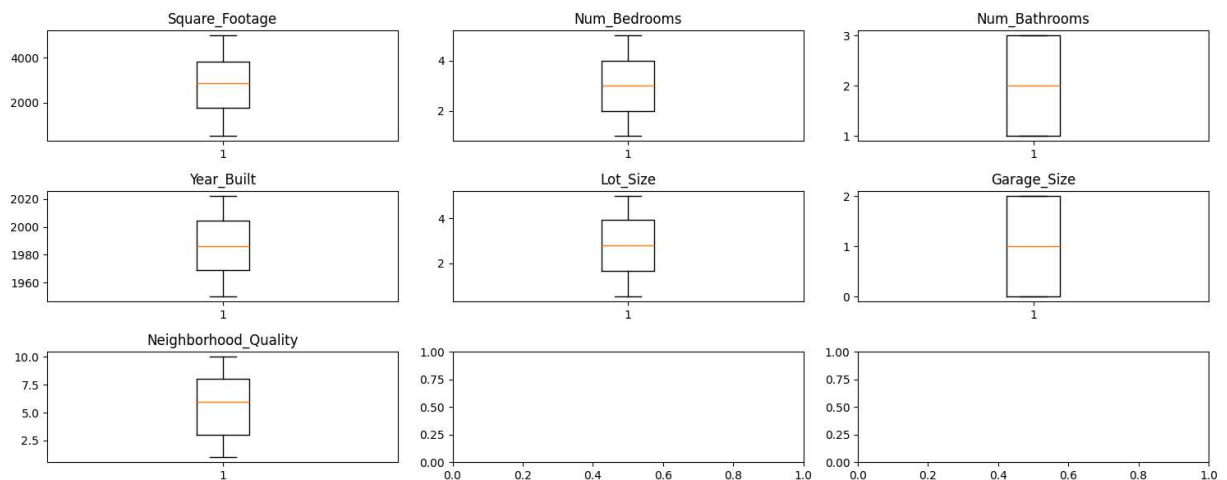
```
In [55]: sns.scatterplot(x='Square_Footage', y='House_Price', data=data, hue='Neighborhood_Q')
```

```
Out[55]: <Axes: xlabel='Square_Footage', ylabel='House_Price'>
```

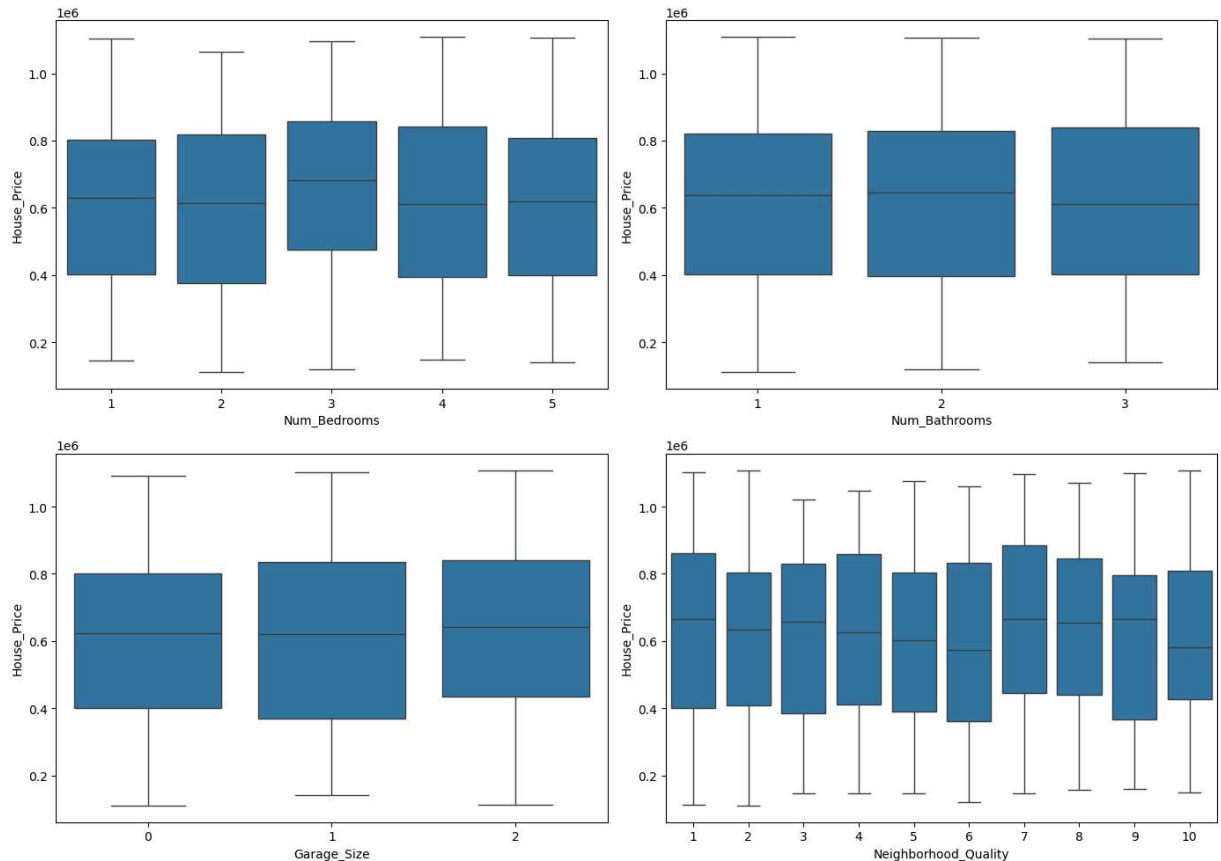



```
In [77]: data_columns = len(X.columns)
col = 3
row = math.ceil(data_columns / col)
fig, axes = plt.subplots(row,col, figsize=(15,6))
axes = axes.flatten()

for i, column in enumerate(X.columns):
    axes[i].boxplot(x=X[column])
    axes[i].set_title(f"{column}")
plt.tight_layout()
plt.show()
```



```
In [74]: fig, axes = plt.subplots(2, 2, figsize=(14,10))
sns.boxplot(x='Num_Bedrooms', y='House_Price', data=data, ax=axes[0,0])
sns.boxplot(x='Num_Bathrooms', y='House_Price', data=data, ax=axes[0,1])
sns.boxplot(x='Garage_Size', y='House_Price', data=data, ax=axes[1,0])
sns.boxplot(x='Neighborhood_Quality', y='House_Price', data=data, ax=axes[1,1])
plt.tight_layout()
```



```
In [78]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
```

```
In [79]: X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

```
In [80]: X.head(2)
```

```
Out[80]:
```

	Square_Footage	Num_Bedrooms	Num_Bathrooms	Year_Built	Lot_Size	Garage_Size	Neighborhood_Quality
0	1360	2	1	1981	0.5996	0	1
1	4272	3	3	2016	4.7530	1	10



```
In [81]: y
```

```
Out[81]: 0      262382.8523
         1      985260.8545
         2      777977.3901
         3      229698.9187
         4     1041740.8589
         ...
        995     701493.9971
        996     683723.1607
        997     572024.0236
        998     964865.2986
        999     742599.2533
        Name: House_Price, Length: 1000, dtype: float64
```

```
In [82]: scaler = StandardScaler()
```

```
In [89]: # scaled X_train, X_test data
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.fit_transform(X_test)
```

```
In [90]: # create model
linear = LinearRegression()
```

```
In [91]: # train model with 'X_train_scaled' and 'y_train' dataset
linear.fit(X_train_scaled, y_train)
```

```
Out[91]: ▾ LinearRegression ⓘ ?
          ▶ Parameters
          ▶ Fitted attributes
```

```
In [92]: # predict the value using X_test_scaled
y_pred = linear.predict(X_test_scaled)
```

```
In [93]: from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score, root
```

```
In [94]: print("R2 score: ", r2_score(y_test, y_pred))
print("Mean squared error: ", mean_squared_error(y_test, y_pred))
print("Root mean squared error: ", root_mean_squared_error(y_test, y_pred))
print("Mean absolute error: ", mean_absolute_error(y_test, y_pred))
```

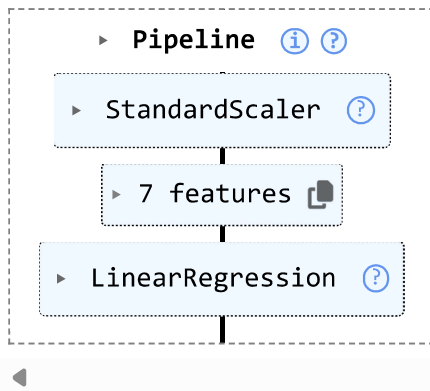
```
R2 score:  0.9980776476391371
Mean squared error:  123912636.09820361
Root mean squared error:  11131.605279482543
Mean absolute error:  9136.432140485094
```

```
In [95]: from sklearn.pipeline import Pipeline
```

```
In [96]: pipeline = Pipeline(steps = [('scaler', StandardScaler()), ('model', LinearRegression)])
```

```
In [97]: pipeline.fit(X_test, y_test)
```

Out[97]:



```
In [99]: y_pred_2 = pipeline.predict(X_test)
y_pred_2[:10]
```

```
Out[99]: array([ 869611.08206301, 489505.48144555, 947831.65654267,
 1037477.38965144, 777521.4639799 , 732687.06439741,
 996881.29179001, 886653.4057679 , 797302.33974542,
 933786.41460365])
```

```
In [101... print("R2 score: ", r2_score(y_test, y_pred_2))
print("Mean squared error: ", mean_squared_error(y_test,y_pred_2))
print("Root mean squared error: ", root_mean_squared_error(y_test,y_pred_2))
print("Mean absolute error: ", mean_absolute_error(y_test,y_pred_2))
```

```
R2 score: 0.99847325989369
Mean squared error: 98411922.32042667
Root mean squared error: 9920.278338858576
Mean absolute error: 7898.025677197687
```

In []: